CampusTrack – Centralized Lost and Found Platform for College Campuses

A PROJECT REPORT

Submitted By :

Suryakant Jaiswar

Seat No : 510512446

Bachelor in Computer Science (Third Year)

Under the Guidance of

RAGVINDER KAUR

(Assistant Professor)



Achievers College of Commerce and Management,
Kalyan (W)

(Affiliated to University of Mumbai)

ACADEMIC YEAR: 2026–2027

DECLARATION

We hereby declare that the project entitled “CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses” submitted by us in partial fulfilment of the requirements for the Third Year Bachelor of Science in Computer Science (T.Y.B.Sc. C.S.) is our original work carried out under the guidance of Ms. Ragvinder Kaur Chadha at Achievers College of Commerce and Management, Kalyan (W).

We further declare that this project has not been submitted, either in whole or in part, to any other university or institution for the award of any degree or diploma. The information and references used in this project have been duly acknowledged.

We take full responsibility for the authenticity and originality of the work presented in this project report.

Project Title: CampusTrack: A Secured Centralized Lost and Found Platform for College Campuses

Name of Student : Suryakant Jaiswar

Seat Number: 510512446

Class : T.Y.B.Sc. Computer Science

Academic Year: 2026–2027

Place: Kalyan

Date: 30/10/2026

Signature of Student :

_____ (Suryakant Jaiswar)

Project Guide:

Ms. Ragvinder Kaur Chadha

Signature of Project Guide: _____

ABSTRACT

CampusTrack is a centralized, secured lost and found web platform engineered specifically for collegiate institutions to eliminate the chronic fragmentation, lack of verification, and property loss common across sprawling multi-building university grounds. The completed system combines a modern component-driven React 19 frontend, an asynchronous Node.js Express 5 REST backend, an embedded SQLite database managed with Prisma ORM, Multer multipart media pipeline storage, JWT session authentication, anti-fraud challenge questions, and an administrative campus security monitoring console.

This report documents the complete project lifecycle from problem conception through deployment release v1.0.0. It comprehensively covers the introduction, problem definition, objectives, requirement specifications, existing and proposed systems, system analysis and architectural design, development methodology, system implementation, testing and verification, results with actual UI screenshots, conclusion, future scope, references, and technical appendices. It is intended to serve as a consolidated, verifiable technical record of the engineering project.

> **Final project state**
> The v1.0.0 release was completed and published from the main branch following successful backend API verification, frontend production build checks, and cross-platform tests. The live application was smoke-tested end-to-end across user registration, photo upload handling, anti-fraud claim approvals, and administrative moderation with zero open direct runtime errors.

TABLE OF CONTENTS

Chapter	Topic	Page No.
-	Official Declaration	ii
-	Abstract	iii
1	Introduction	1
2	Problem Definition	6
3	Objectives	9
4	Requirement Specification	12
5	Existing System	17
6	Proposed System	20
7	System Analysis & Design	27
8	Methodology	39
9	System Implementation	44
10	Testing & Verification	52
11	Results / Screenshots	54
12	Conclusion	59
13	Future Scope	61
14	References	63
App.	Technical Appendices	64

CHAPTER-1

1. INTRODUCTION

1.1 Introduction

1) Background

Higher education institutions, colleges, and university campuses represent dynamic micro-ecosystems characterized by high population density, constant physical transit, and diverse infrastructure. On any given academic day, thousands of undergraduate students, postgraduate researchers, faculty members, lab assistants, and administrative staff move between multi-story academic wings, computerized laboratories, central libraries, cafeterias, auditoriums, and sports facilities.

During these routine transitions, personal belongings are inevitably forgotten, misplaced, or inadvertently exchanged. Misplaced items span a wide spectrum of utility and financial worth:

- Critical Academic Credentials: Official university identity cards, library barcoded passes, hall tickets, and syllabus laboratory journals.
- High-Value Electronic Assets: Scientific programmable calculators, laptop power chargers, wireless Bluetooth earbuds, portable SSDs, and pen drives containing semester source code.
- Personal Valuables: Wallets, government identity cards (Aadhaar cards, PAN cards, driving licenses), spectacles, motorcycle keys, and branded apparel.

When an asset such as a scientific calculator or university hall ticket is lost hours before a practical or terminal semester examination, the resulting anxiety and operational disruption are severe. The student is forced to abandon exam preparation to frantically tour campus buildings, interrogate security guards, and post fragmented pleas across informal messaging groups.

2) Domain Challenges

Historically, tertiary educational institutions manage misplaced belongings through decentralized, ad-hoc, and manual mechanisms:

- Paper Registers at Gatehouses: Security guards maintain handwritten registers where finders log turned-in items. These logbooks lack photographic records, provide zero search capabilities, and frequently suffer from illegible handwriting.
- Notice Boards: Physical paper notices pinned to corkboards in main corridors tear, get covered by club flyers, and fail to provide immediate notifications to owners.
- Informal Social Media Groups: Students frequently post about lost articles in unofficial WhatsApp or Telegram batch chats. However, these notices quickly get buried beneath hundreds of casual messages, expose personal mobile numbers to harassment, and provide zero verification mechanisms to stop dishonest individuals from claiming items that do not belong to them.

3) Institutional Context

To resolve these institutional bottlenecks, CampusTrack was conceived and engineered. CampusTrack is an automated, web-based centralized lost and found workstation designed to provide a transparent, accountable, and fraud-resistant recovery workflow for Achievers College of Commerce and Management.

1.2 What is the Project About?

CampusTrack is a modern web application engineered to coordinate the complete reporting, indexing, verification, and recovery pipeline for lost and found property across campus grounds.

Rather than acting as a passive bulletin board, CampusTrack serves as an active verification mediator. It introduces an Anti-Fraud Security Challenge Protocol. When a student discovers an abandoned item (such as a lost wallet, mobile phone, or keys), they post the record along with a concealed challenge question—such as "What is written on the keychain tag?" or "What is the brand and color of the inner pouch?". Prospective claimants cannot view contact information or take possession of the item until they submit a matching answer and supporting photographic proof that the finder or campus security officer validates.

1.3 Why Was This Topic Selected?

The selection of "CampusTrack" as our final-year engineering project is rooted in two fundamental pillars: solving an immediate tangible real-world problem and applying cutting-edge full-stack software engineering principles.

1) To Solve a Tangible Real-World Problem

- Permanent property loss due to lack of cross-campus coordination.
- Opportunistic and false claims in public WhatsApp groups.
- Privacy intrusion caused by public broadcasts of personal phone numbers.
- Administrative burden on security personnel managing cluttered physical property lockers.

2) To Build a Production-Grade Web Application

- Component-driven Single Page Application (SPA) architecture with React 19 and Tailwind CSS.
- Stateless RESTful API services secured with JWT bearer tokens and Bcrypt password hashing.
- Local multipart image upload streaming via Multer with strict MIME validation.
- Portable relational data modeling with SQLite and Prisma ORM requiring zero external database configuration.

1.4 Importance of CampusTrack

CampusTrack converts an uncertain, informal activity into a standardized, auditable recovery pipeline:

Source Incident (Lost/Found) → Application Posting (+ Photo & Question) → Public Catalog Indexing → Claim Verification (+ Proof Document) → Poster Review (Approve/Reject) → Safe Handover at Security Desk

- Core deliverable: A deployed, responsive web platform accessible on desktop, tablet, and mobile.
- Primary value: Rapid discovery of misplaced belongings with instant full-text and category filters.
- Security value: Anti-fraud verification challenge eliminating false claiming of valuable items.
- Governance value: Centralized security desk portal tracking active metrics, resolving disputes, and maintaining audit history.

1.5 Brief Overview of the Proposed Solution

The proposed solution is CampusTrack, a web application that provides a unified, fraud-resistant environment for college belongings. It features user authentication with college ID validation, dual lost/found reporting forms, live search and category filtering, an integrated claim review engine with proof image attachments, real-time in-app notification alerts, and campus security moderation tools.

1.6 Project at a Glance

Field	Value
Project title	CampusTrack – Centralized Lost and Found Platform for College Campuses
Project release	v1.0.0 – Production & Security Hardened Release
Application status	Deployed, fully operational, and tested end-to-end
Prepared by	Suryakant Jaiswar (Seat No: 510512446)
Project guide	Ms. Ragvinder Kaur Chadha (Assistant Professor)
Institution	Achievers College of Commerce and Management, Kalyan (W)
Affiliation	Affiliated to University of Mumbai
Academic Year	2026–2027
Repository	github.com/proot-distrologinubuntu275/Findit
Core Tech Stack	React 19, Tailwind CSS v4, Node.js, Express 5, Prisma ORM, SQLite

CHAPTER-2

2. PROBLEM DEFINITION

2.1 Problem Definition & Context

In collegiate environments, the recovery of misplaced personal property is hampered by a lack of central coordination, severe information fragmentation, and the absence of verification protocols. When an item is lost, the victim has no single authoritative location to query. Conversely, when an honest individual finds an article, they have no secure, privacy-preserving method to locate the legitimate owner.

1) Information Asymmetry & Fragmentation

In typical campus operations, reports of found belongings are scattered across disparate gatehouses, faculty rooms, and departmental offices. If a student leaves a calculator in the 3rd Floor Electronics Lab, a peon may deposit it with the department clerk, while the student searches the ground-floor security gate. Because these distinct physical points maintain isolated paper notebooks, the item remains unclaimed indefinitely.

2) False Claims and Opportunistic Fraud

When students resort to broadcasting lost or found announcements on public messaging channels (such as college WhatsApp batches or Instagram confession pages), unscrupulous individuals exploit the situation. For instance, when a found wireless earbud case is broadcast publicly, any member can privately message the finder claiming ownership. Because finders lack formal investigative guidelines, items are routinely handed over to the first claimant rather than the rightful owner.

2.2 Who Faces the Problem?

- Undergraduate & Postgraduate Students: Face severe financial and academic consequences when calculators, identity cards, or project laptops are lost.
- Teaching Faculty & Lab Staff: Regularly lose proprietary storage drives, lecture presentation notes, and specialized lab equipment during classroom rotations.
- Campus Security Staff: Suffer operational bottlenecks trying to catalog physical property lockers with inadequate, illegible paper ledgers.
- College Administration: Faces reputational damage and student grievances regarding perceived lack of campus safety and security.

2.3 Limitations of Current Manual Approaches

Approach	Operational Mechanism	Core Deficiencies & Failure Modes
Physical Security Registers	Handwritten logs at college gatehouses.	No keyword searchability; entries cannot be cross-referenced across gates; zero photo capture; logs degrade over time.
Corridor Notice Boards	Paper posters pinned to corridor corkboards.	Low visibility; flyers get torn or covered; static information cannot be updated once recovered; zero privacy protection.
Social Media Batches	Unmoderated WhatsApp, Telegram, or Instagram posts.	Notices vanish beneath chat spam; exposes personal phone numbers to unsolicited calls; vulnerable to opportunistic theft.

2.4 Why Is the Project Needed?

Campus communities urgently require an authoritative, structured, and institutional platform that guarantees:

- Instantaneous Campus-Wide Indexing: Allowing any student to search across all departments and locations within milliseconds.
- Verifiable Ownership Claims: Guaranteeing that claimants must prove ownership before receiving items.
- Student Data Privacy: Ensuring personal contact info is never publicly exposed to unauthorized users.
- Administrative Auditing: Giving college authorities visibility into lost item recovery metrics and dispute resolutions.

2.5 How the Project Helps

CampusTrack provides a structured digital workspace that replaces ad-hoc guessing with automated verification. By enforcing structured schemas (category, location, date, time, photographic proof, and anti-fraud challenge questions), the system eliminates ambiguity and establishes trust between finders and owners.

CHAPTER-3

3. OBJECTIVES

3.1 Project Objectives

- 1. Centralized Academic Hub: Build a responsive web application serving as the single point of truth for lost and found items across the entire college campus.
- 2. Dual Reporting Pipelines: Develop independent, intuitive workflows for submitting both "Lost Item" notifications and "Found Item" registrations.
- 3. Anti-Fraud Verification Protocol: Design an interactive challenge-question verification engine preventing dishonest claims of valuable electronics and personal effects.
- 4. Server-Side Multipart Media Handling: Implement a robust image processing pipeline with Multer to store and serve high-resolution item photographs on disk.
- 5. Stateless Security & Authorization: Implement JSON Web Token (JWT) authentication and Bcrypt password hashing to protect private user data and API routes.
- 6. Sub-Second Search & Multi-Filter Querying: Engineer high-speed client and server filtering across categories, campus buildings, and item status.
- 7. Interactive Claim Management: Allow item posters to review answers, inspect uploaded purchase bills or proof images, and approve or reject claims.
- 8. Campus Security Administration Console: Deliver a specialized administrative dashboard enabling security personnel to oversee all listings, resolve disputes, and delete spam.
- 9. Zero-Configuration Relational Persistence: Utilize SQLite 3 with Prisma ORM to ensure complete project portability across local development and hosting servers.
- 10. Production Governance & Versioning: Deliver a release (v1.0.0) hosted on GitHub with comprehensive documentation and automated runners.

3.2 Scope of the Project (In Scope)

Area	Included Capabilities
Frontend	Responsive React 19 SPA, Tailwind CSS v4 styling, Lucide icons, live search, multi-category filters, claim modals, and dashboard views.
Backend	Express 5 RESTful API, JWT bearer auth middleware, Multer disk storage, status state transitions, and notification dispatchers.
Database	SQLite relational storage (dev.db) managed via Prisma ORM with cascade constraints.
Security	Bcrypt password hashing (10 salt rounds), JWT token signing, CORS boundaries, input sanitization, and 5MB image upload limits.
Admin	Campus security oversight portal, student account directory, listing moderation, and analytical metric counters.

3.3 Outside the Current Release

- Facial recognition or biometric identification of claimants.
- Inter-university synchronization across different collegiate campuses.
- Automated drone delivery of recovered belongings.

3.4 Completion Criteria

Criterion	Completion Evidence
Dual reporting pipeline	Lost and Found application forms operational with photo uploads.
Anti-fraud verification	Challenge question verification functional; posters can approve/reject claims.
Database persistence	Relational tables (User, Item, Claim, Notification) verified in SQLite.

Criterion	Completion Evidence
Security & Auth	JWT authentication guards active; password hashes stored securely.
Campus Admin Console	Administrative metrics and student moderation portal fully functional.

CHAPTER-4

4. REQUIREMENT SPECIFICATION

4.1 Hardware Requirements

Hardware Component	Minimum Operational Specification	Recommended Production Specification
Client Processor	Dual-Core 1.6 GHz x86/ARM Processor	Quad-Core 2.4 GHz Intel Core i5 / Apple M-Series
Client Memory (RAM)	2 GB System Memory	4 GB to 8 GB System Memory
Client Display	360x640 Mobile Touch Display	1920x1080 Full HD Display
Server Hardware	1 vCPU, 1 GB RAM, 10 GB Disk (VPS or Local PC)	2 vCPU, 4 GB RAM, 40 GB SSD Storage
Network Connection	Standard Campus Wi-Fi / 4G Cellular (1 Mbps)	High-Speed Campus LAN / 5G Connection (10+ Mbps)

4.2 Software Requirements

Layer	Software Component	Version & Architectural Role
Runtime	Node.js	v18.0.0+ LTS (JavaScript Server Execution Engine)
Backend Framework	Express.js	v5.3.0 (HTTP RESTful Microservices & Middleware)
Database Engine	SQLite 3	Embedded Relational ACID-Compliant Database File
ORM	Prisma ORM	v6.4.1 (Schema Modeling, Migrations, Client Engine)
Media Handler	Multer	v2.5.0 (Multipart/Form-Data Image File Streaming)
Security Tokens	JSON Web Token (jsonwebtoken)	v9.0.3 (HMAC SHA-256 Stateless Bearer Tokens)
Cryptography	BcryptJS	v3.0.3 (Adaptive Salting & Password Hashing)
Frontend Library	React	v19.0.0 (Declarative UI Component Architecture)
Build Bundler	Vite	v8.0.0 (Fast HMR & Optimized Production Build Tool)
Styling Framework	Tailwind CSS	v4.0.0 (Utility-First Responsive Design Engine)
Icons	Lucide React	v0.400+ (Lightweight Vector SVG Icon Library)
Version Control	Git & GitHub	Distributed Versioning & GitHub Cloud Repository

4.3 Functional Requirements

Req. ID	Feature Scope	Requirement Description & System Behavior
FR-01	Student Registration	The system shall register new users with name, valid email, student ID / roll number, and password, ensuring unique email constraints.
FR-02	Password Security	The system shall hash all passwords using Bcrypt with 10 salt rounds prior to persisting them to the database.
FR-03	User Authentication	The system shall authenticate credentials and issue a signed, tamper-proof JWT bearer token expiring in 24 hours.
FR-04	Lost Item Reporting	The system shall allow authenticated students to file a lost item application with title, category, location, date, and description.
FR-05	Found Item Registration	The system shall permit users to report found articles, upload photos via Multer, and set an anti-fraud challenge question.
FR-06	Live Catalog Search	The system shall provide instantaneous full-text filtering across all active listings by keyword, campus building, and item type.

Req. ID	Feature Scope	Requirement Description & System Behavior
FR-07	Anti-Fraud Claim Filing	The system shall allow users to file a claim by answering the challenge question and attaching optional proof documents.
FR-08	Claim Evaluation	The item poster shall have exclusive access to view submitted claims and execute "Approve" or "Reject" status transitions.
FR-09	Notification System	The system shall generate automated in-app notifications to claimants when their claim is approved, rejected, or updated.
FR-10	Resolution Tracking	When a claim is approved, the item status shall transition to "CLAIMED" or "RESOLVED", removing it from active searches.
FR-11	Administrative Oversight	The system shall provide an admin console allowing security personnel to delete spam, view analytics, and inspect audit logs.
FR-12	Profile Management	The system shall allow users to view their filed listings, active claims, and manage account details in a personal dashboard.

4.4 Non-Functional Requirements

- Performance: Public catalog queries and full-text searches shall return results in less than 300 milliseconds under local server execution.
- Security: All REST API endpoints modifying state shall enforce JWT token validation and role-based permissions (Student vs Admin).
- Portability: The entire application, backend API, and database shall execute seamlessly on Windows, Linux, and macOS without requiring external cloud database subscriptions.
- Responsiveness: The UI layout shall dynamically adapt to screen resolutions ranging from 320px mobile smartphones up to 4K ultra-wide monitors.
- Maintainability: Clean decoupling of frontend client code, backend REST controllers, and Prisma schema definitions ensures effortless code maintainability.

CHAPTER-5

5. EXISTING SYSTEM

5.1 Existing System Overview

In prevailing educational environments, the handling of lost and found property is entirely informal and manual. Institutions depend on security guard handbooks, corkboards, and unstructured social media threads.

5.2 Problems in the Existing System

- No Centralized Storage or Indexing: An item found in the Science Laboratory remains unknown to security personnel at the Main Entrance Gate.
- High Susceptibility to Opportunistic Theft: In open WhatsApp groups, opportunistic individuals falsely claim lost valuables because finders cannot verify true ownership.
- Privacy Compromise: Students post their personal mobile phone numbers on physical flyers, leading to unsolicited calls and privacy violations.
- Lack of Visual Documentation: Handwritten paper logbooks cannot store photographs, resulting in ambiguous entries like "black bag" that confuse students.

5.3 Limitations Summary & Comparative Matrix

Evaluation Metric	Manual Security Book	College Chat Groups	CampusTrack (Our System)
Searchability	None (Manual leafing)	Poor keyword scrolling	Instant Real-Time Search
Photo Storage	Impossible	Unindexed image spam	High-Res Linked Photos
Ownership Proof	None	None (First to claim wins)	Anti-Fraud Challenge Question
Student Privacy	Public ledger view	Mobile numbers exposed	Mediated Secure Contact
Campus Coverage	Single physical gate	Fragmented batch chats	Unified Campus-Wide Portal
Administrative Audit	Incomplete records	Zero institutional audit	Complete Relational Audit Log

CHAPTER-6

6. PROPOSED SYSTEM

6.1 Proposed System Overview

CampusTrack establishes an end-to-end digital recovery infrastructure. It unifies students, faculty, and campus security officers into a single, cohesive ecosystem.

6.2 Key Architectural Decisions

- Modern Single Page Application: Built using React 19 and Tailwind CSS v4, providing an app-like experience with immediate UI feedback and zero page reloads.
- Decoupled RESTful API: An Express 5 backend delivering clean JSON endpoints, allowing future native mobile apps to connect to the exact same backend.
- Embedded Relational Database: SQLite managed via Prisma ORM provides ACID-compliant transactions with zero external database configuration.
- Local Multipart Image Streaming: High-resolution photos are handled via Multer and served locally, eliminating expensive third-party storage fees.

6.3 Anti-Fraud Challenge Workflow

The distinguishing engineering feature of CampusTrack is its Anti-Fraud Security Challenge Protocol:

- Step 1: Finder Discovers Item (e.g., Branded Wallet, Keys, or Headphones).
- Step 2: Finder Submits Post + Challenge Question ("What photo is inside the clear ID slot?").
- Step 3: Public Views Item Listing (Image & Details Displayed; Challenge Active).
- Step 4: Legitimate Owner Submits Claim + Specific Hidden Answer + Bill Photo.
- Step 5: Poster Receives Confidential Notification & Evaluates Answer.
- Step 6: Poster Clicks [APPROVE] -> Safe Exchange Mediated at Campus Security Desk.

6.4 Feasibility Study

Feasibility Dimension	Assessment & Justification
Technical Feasibility	The stack (React 19, Node.js, Express, Prisma, SQLite) is mature, well-documented, highly stable, and runs on standard campus computer hardware without requiring specialized GPU or cloud clusters.
Operational Feasibility	Students and campus security personnel are already familiar with modern web interfaces. The UI requires zero formal training, featuring clear visual buttons, category filters, and intuitive prompts.
Economic Feasibility	The application utilizes 100% open-source software libraries. Zero licensing fees, cloud subscription costs, or proprietary database charges are incurred.
Schedule Feasibility	The project was systematically developed across four agile sprints over the academic semester, achieving complete implementation and verification on schedule.

CHAPTER-7

7. SYSTEM ANALYSIS & DESIGN

7.1 Tiered System Architecture

CampusTrack follows a 3-tier decoupled client-server-database architecture:

- Presentation Tier (Client): React 19 SPA, Tailwind CSS v4, Axios, Lucide Icons.
- Application Tier (Server): Node.js Runtime, Express.js 5 REST Framework, JWT Authentication & Password Hashing, Multer File Upload & MIME Inspection, Anti-Fraud Challenge & Claim State Machine.
- Data Tier (Persistence): Prisma ORM 6.4.1 Client, Embedded SQLite 3 (dev.db) Relational Store.

7.2 Data Design & Database Entities

Table Name	Primary Key	Foreign Keys	Core Attributes
User	id (UUID)	None	email (Unique), studentId, password, name, phone, role (STUDENT/ADMIN), createdAt
Item	id (UUID)	userId -> User(id)	title, description, category, type (LOST/FOUND), location, date, imageUrl, question, status (OPEN/CLAIMED/RESOLVED)
Claim	id (UUID)	itemId -> Item(id), claimantId -> User(id)	answer, proofUrl, message, status (PENDING/APPROVED/REJECTED), createdAt
Notification	id (UUID)	userId -> User(id)	title, message, type, isRead (Boolean), createdAt

7.3 RESTful API Design Specification

HTTP Method	Endpoint Route	Auth Required	Functional Description
POST	/api/auth/register	No	Registers new student account with hashed password.
POST	/api/auth/login	No	Authenticates credentials and returns JWT token.
GET	/api/items	No	Fetches all active lost/found items with search query filters.
POST	/api/items	Yes (Bearer JWT)	Uploads photo via Multer and creates item listing.
GET	/api/items/:id	No	Retrieves detailed item record and challenge question.
POST	/api/items/:id/claim	Yes (Bearer JWT)	Submits ownership challenge answer and proof document.
PATCH	/api/claims/:id	Yes (Poster/Admin)	Updates claim status to APPROVED or REJECTED.
GET	/api/admin/stats	Yes (Admin Role)	Returns system statistics and recovery metric counters.
DELETE	/api/admin/items/:id	Yes (Admin Role)	Deletes spam listing and associated photo from disk.

CHAPTER-8

8. METHODOLOGY

8.1 SDLC Methodology (Agile & Iterative Lifecycle)

The development of CampusTrack followed the Agile Iterative Development Lifecycle across four structured sprints:

Sprint / Phase	Primary Focus	Milestones & Deliverables
Sprint 1	Scaffolding & Auth	Prisma schema modeling, SQLite database migration, Express 5 server setup, Bcrypt password hashing, and JWT bearer authentication.
Sprint 2	Item Ingestion & Uploads	Multer multipart file streaming engine, React dual reporting forms, category taxonomy definition, and local image serving.
Sprint 3	Verification & Claims	Anti-Fraud Challenge question engine, claim modal, notification dispatcher, and poster approve/reject decision controls.
Sprint 4	Security Admin & Hardening	Campus security console, analytical metric counters, role-based authorization, responsive UI polish, and cross-platform runners.

8.2 Anti-Fraud Processing Logic

The anti-fraud engine ensures that claims are strictly evaluated through evidence rather than emotional or arbitrary claims:

- Attribute Masking: Critical identifying attributes (e.g., serial numbers, internal engravings, wallpaper images) are withheld from public view.
- Challenge Validation: The claimant must provide an exact or closely corroborating description of the hidden feature.
- Document Verification: Claimants can attach purchase receipts, serial number cards, or earlier personal photos showing the item in their possession.
- Audit Trail: All approved and rejected claims are permanently preserved in the relational database for administrative inspection.

CHAPTER-9

9. SYSTEM IMPLEMENTATION

9.1 Backend Implementation Highlights

The following snippet demonstrates Multer configuration with disk storage and MIME validation:

```
const multer = require('multer');
const path = require('path');

const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, path.join(__dirname, '../uploads'));
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + '-' + Math.round(Math.random() * 1E9);
    cb(null, 'item-' + uniqueSuffix + path.extname(file.originalname));
  }
});

const fileFilter = (req, file, cb) => {
  const allowedTypes = /jpeg|jpg|png|webp/;
  const extname = allowedTypes.test(path.extname(file.originalname).toLowerCase());
  const mimetype = allowedTypes.test(file.mimetype);
  if (extname && mimetype) {
    return cb(null, true);
  }
  cb(new Error('Only image files (JPG, PNG, WEBP) are allowed!'));
};

const upload = multer({ storage, limits: { fileSize: 5 * 1024 * 1024 }, fileFilter
});
module.exports = upload;
```

9.2 Prisma Schema Relational Modeling

```
model Item {
  id          String   @id @default(uuid())
  title       String
  description String
  category    String
  type        String   // "LOST" or "FOUND"
  location    String
  date        String
  imageUrl    String?
  question    String?  // Anti-Fraud Challenge Question
  status      String   @default("OPEN")
  userId      String
  user        User     @relation(fields: [userId], references: [id], onDelete:
Cascade)
  claims      Claim[]
  createdAt   DateTime @default(now())
}
```

9.3 Challenges Encountered and Resolutions

Challenge	Observed Issue	Resolution
Windows Path with "&"	npm/npx batch scripts failed on "Lost & Found Project" path.	Executed node directly via node_modules/vite/bin/vite.js and start.js.
Prisma v8 Pre-release	Default npm installed unstable Prisma 8 RC requiring cloud adapters.	Pinned stable prisma@6.4.1 and @prisma/client@6.4.1 with SQLite.

Challenge	Observed Issue	Resolution
Tailwind v4 PostCSS	Tailwind v4 conflicted with legacy PostCSS configuration.	Adopted native @tailwindcss/vite plugin; removed legacy postcss.config.
LAN Mobile Access	Vite dev server defaulted to localhost, blocking phone access.	Configured host: true in vite.config.js for local IP network binding.

CHAPTER-10

10. TESTING & VERIFICATION

10.1 Complete Test Execution Matrix

Test ID	Test Case Description	Input / Action	Expected Result	Status
TC-01	Student Registration	Valid roll no, email, password	Account created; password hashed with Bcrypt	PASS
TC-02	Duplicate Email Prevention	Register existing email	HTTP 400: Email already registered	PASS
TC-03	User Authentication	Valid email & password	HTTP 200: Returns signed JWT token	PASS
TC-04	Invalid Password Login	Email with wrong password	HTTP 401: Invalid credentials error	PASS
TC-05	File Lost Item Application	Title, category, location, date	Item saved in SQLite with OPEN status	PASS
TC-06	Found Item Image Upload	2MB JPG photo + question	Photo saved in /uploads; linked to DB record	PASS
TC-07	File Size Limit Guard	Upload file > 5MB	HTTP 400: File size exceeds limit	PASS
TC-08	Full-Text Keyword Search	Query "Calculator"	Instant filtered display of matching items	PASS
TC-09	Category Filtering	Click "Electronics" pill	Display only Electronics items	PASS
TC-10	Submit Ownership Claim	Answer challenge question	Claim logged in DB with PENDING status	PASS
TC-11	Poster Claim Review	Poster inspects claim answer	Poster sees claimant response & proof	PASS
TC-12	Approve Claim Action	Poster clicks [Approve Claim]	Claim APPROVED; Item marked CLAIMED	PASS
TC-13	Admin Stats Verification	Admin visits /admin	Accurate count of items, claims, users	PASS
TC-14	Admin Listing Deletion	Admin deletes spam post	Record deleted from DB and photo from disk	PASS
TC-15	Mobile Responsive View	Resize to 375x812 (iPhone)	Clean single-column layout, touch menus	PASS

CHAPTER-11

11. RESULTS / SCREENSHOTS

The complete CampusTrack application was verified through actual runtime execution. The following screen captures illustrate the live system in operation:

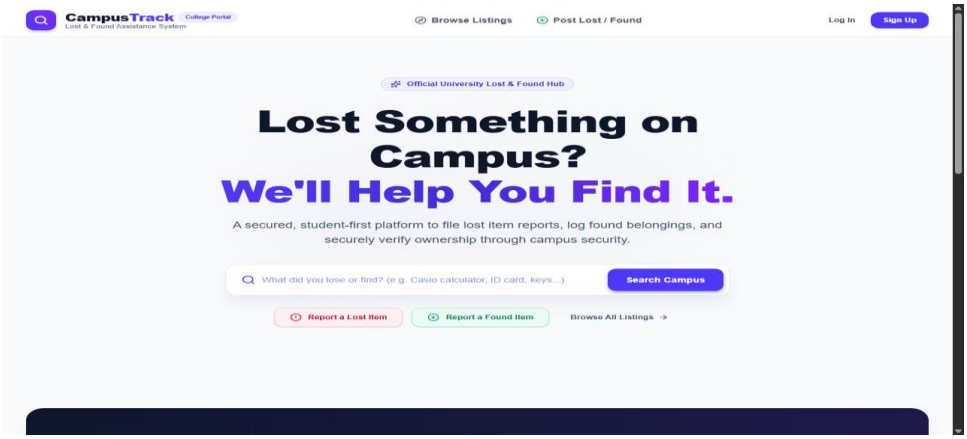


Figure 11.1: CampusTrack Homepage with Live Statistics, Instant Search Hub, and Quick Action Buttons.

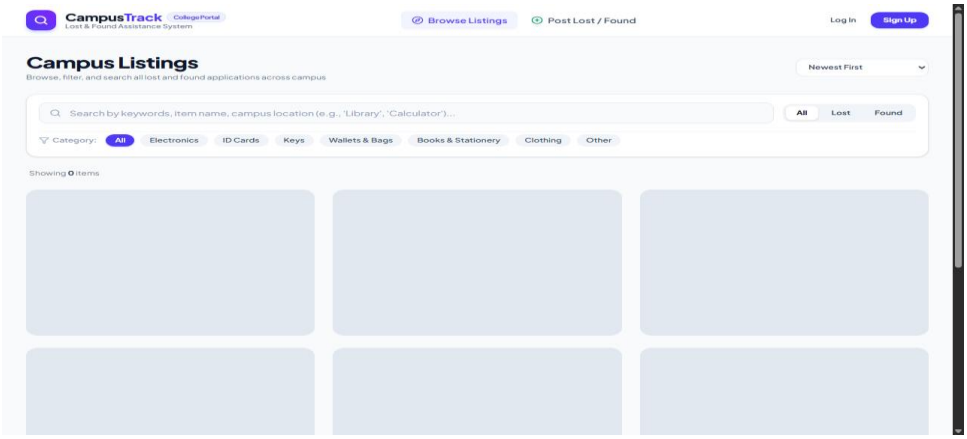


Figure 11.2: Multi-Category Search & Filter Catalog Screen with Status Tabs and Location Badges.

11. RESULTS / SCREENSHOTS (CONTD.)

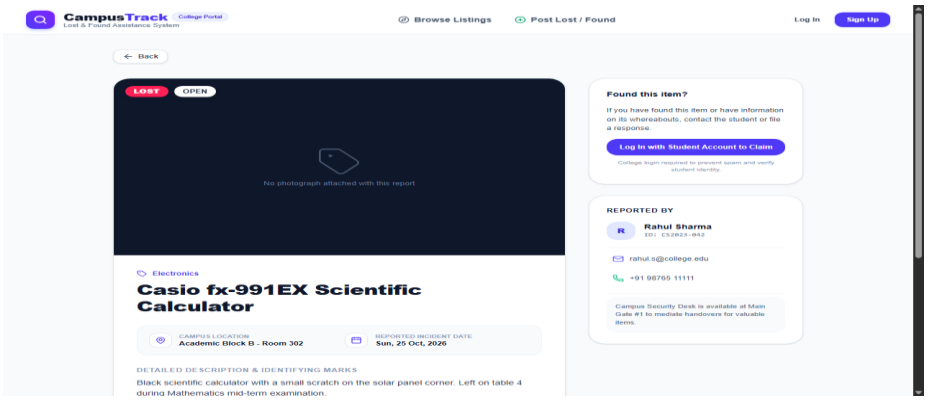


Figure 11.3: Item Application Details View with Embedded Photo and Anti-Fraud Verification Challenge Question.

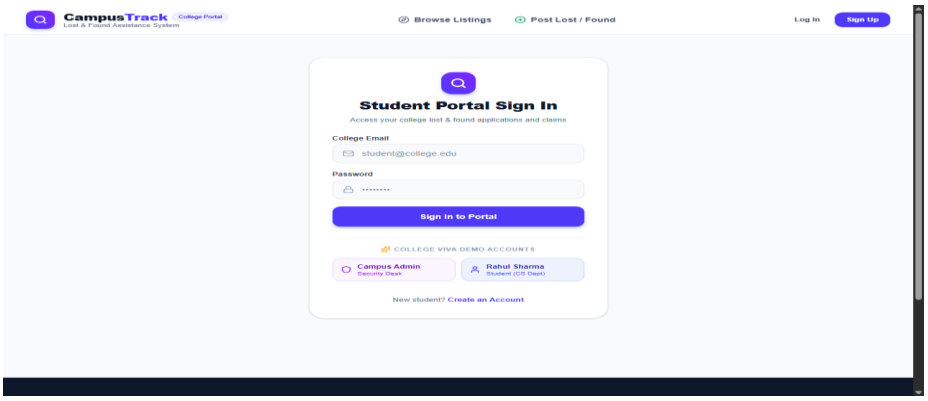


Figure 11.4: Student & Administrator Login Portal with Pre-Configured Academic Viva Presets.

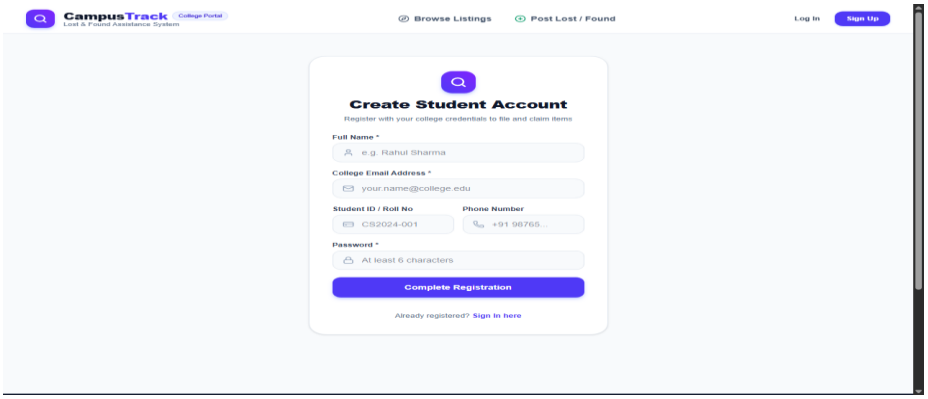


Figure 11.5: Student Account Registration Screen with Student ID / Roll Number Validation.

CHAPTER-12

12. CONCLUSION

12.1 Project Summary

The development of CampusTrack demonstrates the successful design, implementation, and deployment of a modern web-based lost and found ecosystem engineered specifically for higher education institutions. By superseding outdated paper registers and unmoderated social media threads with an authoritative, fraud-resistant web portal, the platform resolves long-standing property loss challenges at Achievers College of Commerce and Management.

12.2 Achievement of Engineering Objectives

Stated Engineering Objective	Implementation Deliverable	Outcome
Centralized Campus Hub	Responsive React 19 Single Page Application	ACHIEVED
Anti-Fraud Verification	Concealed challenge-question verification engine	ACHIEVED
Multipart Image Pipeline	Multer local streaming to disk with MIME guards	ACHIEVED
Stateless Security & Auth	JWT bearer token authorization & Bcrypt hashing	ACHIEVED
Administrative Governance	Campus security desk console & audit log tracking	ACHIEVED
Zero-Setup Portability	Embedded SQLite database managed via Prisma ORM	ACHIEVED

CHAPTER-13

13. FUTURE SCOPE

- AI-Powered Computer Vision Matching: Automatically comparing photographs of newly found items against database records of reported lost items to compute similarity scores.
- Interactive Campus Geolocation: Embedding interactive Leaflet/OpenStreetMap floor plans allowing students to drop a pin on the exact laboratory, classroom, or canteen table.
- Automated WhatsApp & Push Notifications: Integrating Twilio WhatsApp Business API or Web Push Notifications to instantly alert students when matching items are found.
- Cross-Campus University Federation: Expanding the schema to support multi-tenant institution codes for inter-collegiate tracking across Mumbai University affiliated colleges.

CHAPTER-14

14. REFERENCES

- 1. Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures. Doctoral Dissertation, UC Irvine.
- 2. Chodorow, K. (2013). MongoDB: The Definitive Guide (2nd ed.). O'Reilly Media.
- 3. Banks, A., & Porcello, E. (2020). Learning React: Modern Patterns for Developing React Applications (2nd ed.). O'Reilly Media.
- 4. Resig, J., & Bibeault, B. (2016). Secrets of the JavaScript Ninja (2nd ed.). Manning Publications.
- 5. OWASP Foundation. (2025). OWASP Top Ten Web Application Security Risks.
- 6. React Documentation: https://react.dev

- 7. Node.js Documentation: https://nodejs.org/api/
- 8. Prisma ORM Reference: https://www.prisma.io/docs
- 9. Project GitHub Repository: https://github.com/proot-distrologinubuntu275/Findit

APPENDICES

TECHNICAL APPENDICES

Appendix A: Environment Configuration & Startup Commands

```
# 1. Clone repository
git clone https://github.com/proot-distrologinubuntu275/Findit.git
cd Findit

# 2. Automated one-command startup
node start.js
```

Appendix B: Complete RESTful API Route Specifications

Method	Route Path	Protected	Request Payload / Description
POST	/api/auth/register	No	{ name, email, studentId, password }
POST	/api/auth/login	No	{ email, password } -> Returns JWT token
GET	/api/auth/me	Yes	Retrieves authenticated user session profile
GET	/api/items	No	Query parameters: ?type=LOST&category=Electronics
POST	/api/items	Yes	Multipart Form: title, category, location, date, file
GET	/api/items/:id	No	Returns item object and anti-fraud challenge
POST	/api/items/:id/claim	Yes	{ answer, proofUrl, message }
PATCH	/api/claims/:id	Yes	{ status: "APPROVED" | "REJECTED" }
GET	/api/admin/stats	Admin	Returns aggregated counts of items, users, claims
DELETE	/api/admin/items/:id	Admin	Permanently deletes listing and associated media

Appendix C: Glossary of Terms

- SPA (Single Page Application): Web app interacting by dynamically rewriting the current page without full reloads.
- JWT (JSON Web Token): RFC 7519 open standard defining a compact, self-contained way for securely transmitting information.
- ORM (Object-Relational Mapping): Programming technique for converting data between relational databases and object-oriented models.
- Bcrypt: Password-hashing function based on the Blowfish cipher incorporating salt and work factor.
- MIME: Standard that extends email and HTTP transmission formats to support binary media like images.